

Program 8: Addition of Two Polynomials using Linked Lists

Aim

To write a C program that adds two polynomials represented as linked lists.

Theory

Each polynomial is stored as a linked list of (coefficient, exponent) nodes in **descending order of exponent**.

Addition works like **merging two sorted lists**. With pointers p and q on the two polynomials: if $\text{exp}(p) > \text{exp}(q)$, copy the term of p; if $\text{exp}(p) < \text{exp}(q)$, copy the term of q; if the exponents are equal, add the coefficients and store the term only if the sum is non-zero.

Remaining terms of either polynomial are appended to the result.

Time complexity is $O(m + n)$ for polynomials with m and n terms.

Algorithm

1. Read both polynomials and store them in descending order of exponent.
2. Set p and q to the heads of the two lists; create an empty result list.
3. While both are non-NULL, compare exponents and copy / add terms as described in the theory.
4. Copy the remaining terms of p or q.
5. Display P1, P2 and their sum.

C Source Code

```
/*
 * Program : Addition of Two Polynomials using Linked Lists
 *
 * THEORY:
 * - Each polynomial is stored as a linked list of (coefficient, exponent)
 *   nodes in DESCENDING order of exponent.
 * - Addition works like merging two sorted lists using pointers p and q:
 *   exp(p) > exp(q) : copy term of p, advance p
 *   exp(p) < exp(q) : copy term of q, advance q
 *   exp(p) == exp(q) : add the coefficients; store the term only if the
 *                       sum is non-zero; advance both
 * - Remaining terms of either list are then appended.
 * - Time complexity:  $O(m + n)$  for polynomials with m and n terms.
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coef;
    int exp;
    struct Node *next;
};

struct Node *insertTerm(struct Node *head, int coef, int exp) { /* sorted insert */
    if (coef == 0) return head;
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->coef = coef; n->exp = exp; n->next = NULL;

    if (head == NULL || exp > head->exp) { n->next = head; return n; }
    struct Node *t = head;
    while (t->next != NULL && t->next->exp > exp) t = t->next;
```

```

        if (t->exp == exp)          { t->coef += coef; free(n); }
        else if (t->next != NULL && t->next->exp == exp) { t->next->coef += coef; free(n); }
        else { n->next = t->next; t->next = n; }
        return head;
    }

    struct Node *appendTerm(struct Node *head, struct Node **tail, int coef, int exp) {
        struct Node *n = (struct Node *)malloc(sizeof(struct Node));
        n->coef = coef; n->exp = exp; n->next = NULL;
        if (head == NULL) head = n;
        else (*tail)->next = n;
        *tail = n;
        return head;
    }

    struct Node *addPoly(struct Node *p, struct Node *q) {
        struct Node *res = NULL, *tail = NULL;
        while (p != NULL && q != NULL) {
            if (p->exp > q->exp)      { res = appendTerm(res, &tail, p->coef, p->exp); p = p->next; }
            else if (p->exp < q->exp) { res = appendTerm(res, &tail, q->coef, q->exp); q = q->next; }
            else {
                int sum = p->coef + q->coef;
                if (sum != 0) res = appendTerm(res, &tail, sum, p->exp);
                p = p->next; q = q->next;
            }
        }
        for (; p != NULL; p = p->next) res = appendTerm(res, &tail, p->coef, p->exp);
        for (; q != NULL; q = q->next) res = appendTerm(res, &tail, q->coef, q->exp);
        return res;
    }

    void printPoly(struct Node *head) {
        if (head == NULL) { printf("0\n"); return; }
        for (struct Node *t = head; t != NULL; t = t->next) {
            int c = t->coef;
            if (t == head) { if (c < 0) printf("-"); }
            else           { printf(c < 0 ? " - " : " + "); }
            if (c < 0) c = -c;
            if (t->exp == 0) printf("%d", c);
            else {
                if (c != 1) printf("%d", c);
                printf("x");
                if (t->exp != 1) printf("^%d", t->exp);
            }
        }
        printf("\n");
    }

    struct Node *readPoly(int number) {
        struct Node *poly = NULL;
        int n, coef, exp;
        printf("Enter number of terms in polynomial %d: ", number);
        scanf("%d", &n);
        for (int i = 1; i <= n; i++) {
            printf("  Term %d (coefficient exponent): ", i);
            scanf("%d %d", &coef, &exp);
            poly = insertTerm(poly, coef, exp);
        }
        return poly;
    }

    void freePoly(struct Node *h) {
        while (h != NULL) { struct Node *t = h; h = h->next; free(t); }
    }

    int main() {
        struct Node *p1 = readPoly(1);
        struct Node *p2 = readPoly(2);
        struct Node *sum = addPoly(p1, p2);

        printf("\nPolynomial 1 : "); printPoly(p1);
    }

```

```

printf("Polynomial 2 : "); printPoly(p2);
printf("Sum          : "); printPoly(sum);

freePoly(p1); freePoly(p2); freePoly(sum);
return 0;
}

```

Sample Input

Input: $P1 = 5x^3 + 4x^2 + 2$ (3 terms), $P2 = 3x^3 - 4x^2 + 6x + 1$ (4 terms).

Sample Output (program run with the input above)

```

Enter number of terms in polynomial 1:   Term 1 (coefficient exponent):   Term 2 (coefficient exponent)
Polynomial 1 : 5x^3 + 4x^2 + 2
Polynomial 2 : 3x^3 - 4x^2 + 6x + 1
Sum          : 8x^3 + 6x + 3

```